

Sorcery Design

Umar Mustafa, Owen Zhang, Bokang Mogomotsi

Introduction

For Sorcery, we generally followed the recommended order of steps at the end of the provided document to complete the project. At the beginning of constructing the codebase, we started with basic header files according to the classes and methods that we outlined in our UML diagram. Then, we attempted to implement these classes while sticking to the plan of attack, although we had to make a few design modifications along the way. While developing this project, we kept several SOLID design principles in mind, such as the Single Responsibility Principle and the Open/Closed Principle. Our project should satisfy all of the game logic requirements in the Sorcery project specification. Additionally, the design choices within our implementation support the possibility of various changes in the project specification.

Overview

At a high level, our project follows the MVC architecture, reducing coupling by dividing responsibilities amongst the Game, View, and GameController. Each aspect of these individual classes cooperates to do one thing, satisfying the Single Responsibility Principle. The Game class manages the game state, the View class displays aspects of the game, and the GameController processes user input. Hence, this is an implementation with high cohesion as well. We made GameController own the pointers to the Game and View objects because it interacts with the user on the highest level. All owns-a relationships use `std::unique_ptr`, raw pointers are non-owning back references only, eliminating manual delete and leaks.

Game owns pointers to two Player objects, with each Player owning pointers to Deck, Hand, Board, and Graveyard. Deck is the deck of cards the player draws from, Hand contains the cards in the player's hand, Board contains the minions and ritual that the player has in play, and Graveyard contains the player's dead minions. Hand, Board, and Graveyard are all collections of cards that we need to add to and remove from. So, we created a CardCollection parent class for these three classes to share common logic while differing only in domain rules (e.g Board size limit, one ritual)

Each Card is either a Spell, Minion, Enchantment, or Ritual. Every Card has common fields such as a name, a description, and a cost. They also must implement the

`Card::use()` method in some way to define what it means to play or use that card. Since `Card` is an abstract class, it satisfies the Open/Closed Principle since we can make as many `Card` types as we want, saving us a lot of time in the future. `CardFactory` abstracts creation of concrete cards by name through a map from name to a function which initializes the `Card`.

Abilities are also split between activated abilities and triggered abilities. Both types are pretty similar in that they can be “used” in some way and can be owned by a `Minion`. Hence, we created an abstract class `Ability` with subclasses `ActivatedAbility` and `TriggeredAbility`, ensuring that both implement the `Ability::use()` method.

For triggered abilities, the four possible triggers are encoded as `ENUM` values in our codebase. When each of the triggers occurs within the game state, the `Game` raises global events (start/ end turn, enter/leave player) with the associated enum value on all possible rituals and minions in APNAP order, calling `use` on the triggered ability if the triggers match.

Commands are a significant aspect of our game logic. When each `Card` or `Ability` is “used”, it creates a `Command` object and returns it to the `Game`. Again, to satisfy the Open/Closed Principle, we made `Command` an abstract class so that we can create as many `Command` objects as we want. `Game` will call `Command::execute(Game& game)`, which executes the game logic for using the `Card` or `Ability` and mutates the model then notifies the controller (observer).

Invalid user actions throw exceptions caught at the controller level and no state is changed prior to validation, providing a strong exception guarantee.

Design

The first challenge that we encountered in our project was how we were going to address the possible invalid inputs that our program could receive from the user. For example, the user could request to use a `Minion` at a position on the board that does not exist. We knew that in cases like these where the game receives invalid input, the game state should remain unchanged. Hence, we used our knowledge from the course to implement exceptions with strong guarantees. If some class method within the game logic ever received input that was invalid, it would throw an exception that would go through stack unwinding to end back up at our `GameController`, which would catch this exception and communicate it to the user. We made sure that none of the game state gets edited before an exception is thrown to provide a strong guarantee.

Another challenge we encountered was figuring out how to differentiate the many subclasses of Card (Spell, Minion, etc.) within Player. This is because we had a `Player::playCard()` method that would play the card at a specified index in the active player's hand. All of the 4 card types exist in different states after being played. Minions and Rituals get added to the player's Board. Spells disappear and should be deleted from memory. Enchantments get applied onto a Minion according to the Decorator Pattern. Because our method had to deal with each Card type differently, and at compile-time it is not clear what type of Card is being accessed in the player's Hand, we decided to solve this problem using dynamic casting. By using this technique, we could take a Card from Hand and then dynamically cast it into its Card type inside a couple of conditional statements to decide how the game should deal with it. Another part of our code where dynamic casting was useful was the `TextualDisplay` class, which is a concrete class that displays our game state. From a visual standpoint, each Card type has a slightly different appearance. For example, Minions have attack and defense values in their bottom corners, which is something that Spells and Rituals lack. So, in `TextualDisplay::showHand(const Hand& hand)`, we dynamically casted the Cards inside the Hand to their Card types, ensuring that we could access all of the necessary fields for each Card and display the Cards accurately.

An additional design challenge was in maximizing code reusability between abstract and concrete classes. We were able to accomplish this using virtual methods, including virtual destructors when no other methods could be set as pure virtual. Any function that would have a different implementation in a concrete class we set as pure virtual, implementing the other methods within the abstract class. We also took advantage of operator overloading to enhance readability, especially due to the overlap within play/use commands when there are no targets vs multiple targets. This made our code much more intuitive especially when developing game logic within the controller.

Initially, we used decorators for increasing damage and defence only for enchantments and directly updated attack and defence values through spells and abilities. This led to inconsistencies in final attack values. We decided to consolidate all damage and defence modifiers through decorators on the base minion, displaying only the base minion to the user. By doing this, we were able to hold all the logic for minion modifications in one place, improving our process for debugging. This meant that at any place within the code where we were calling pointers to minions, we had to loop through all enchantments to get to the base minion in order to avoid segmentation faults. This also meant we had to differentiate between in game enchantments and non in game enchantments we created to facilitate damage and defence modifications. It also

meant when comparing raw minion pointers, we had to go down the enchantment chain to the base minion before comparison.

The final design challenge that we had to address was memory management. We wanted to make managing memory as simple as possible because memory leaks can arise if, for example, we forget to delete a Card inside a destructor. Another potential issue is double deletion, which can cause undefined behaviour during development. In order to simplify memory management, we opted to use unique pointers for every “owns-a” relationship in our UML diagram. This simplified our workflow because we did not have to manually implement a destructor once in our code. Smart pointers would automatically be deleted when they went out of scope. So, they were very reliable in ensuring that none of our memory leaked.

Resilience to Change

As mentioned before, our project made use of the MVC pattern at the highest level by having a GameController, Game, and View class. This design choice is resilient to change because it cleanly separates the concerns of the game state (Game), the display interface (Model), and the processing of user input (GameController). Note that this promotes high cohesion because each component has a single thing it needs to manage (Single Responsibility Principle). It also promotes low coupling because changes to one of the components does not have much impact on the other components. For example, if the project specification updated and required something different for the display, we would not have to significantly change the GameController or Model because the View is the only component managing the display. So, using the MVC model makes it easier for the high level aspects of the codebase to be maintained and extended.

Our project is also resilient to change through our consistent usage of abstract classes with concrete subclasses throughout the entire codebase. High level classes such as Player were reliant on an abstract Card class whose descendant classes were concrete (an example of the Dependency Inversion Principle). For example, Minion is an abstract subclass of Card. Under Minion are concrete classes that implement each Minion (e.g. Air Elemental). This is an example of high cohesion because every concrete class implements a single Minion’s features. If we wanted to add a new Minion to the game, we could easily add a new concrete class and give it the features shared amongst all Minions. We also do not need to modify any code in the abstract Minion class. Additionally, if the project specification for Minions were to change, we could change

just the Minion class to have changes apply to all concrete Minions, demonstrating low coupling. Some other aspects of our project that are extensible through abstraction are Commands, Abilities, and Views. We can easily create new concrete versions of these abstract classes while making very minimal changes to the high level code.

We implemented the Command Pattern to make each game action a standalone command object (high cohesion). In our project, there is an abstract Command class containing the pure virtual method `Command::execute()`. Each concrete Command will use this method to make some change to the game state. For example, the Novice Pyromancer's activated ability instantiates a `DamageMinionCommand` whose `execute()` method applies one damage to a target minion. This promotes reusability because if we want to implement another Card or Ability that applies damage to a target minion, we can reuse this concrete Command. The invoker, which is the Game class in our case, interacts only with the `Command::execute()` method and is not reliant on its implementation. Hence, this is low coupling and our usage of the Command Pattern means that high level classes do not need to be modified significantly if the project specification requires new game actions.

Additionally, Enchantments were implemented using the Decorator Pattern. Enchantment is a subclass of Minion, with each Enchantment owning a pointer to another Minion. This enabled additional properties to be added to concrete Minions without having to alter the underlying class (low coupling). Hence, if the project specification required Enchantments that modify Minions differently, we would not have to change the Minion class significantly (Open/Closed Principle). Also, each Enchantment is its own Decorator class and performs a single modification on a Minion, which is an example of high cohesion. So, the usage of the Decorator Pattern makes Enchantments very resilient to change.

Finally, the Factory Method was used to define an interface for creating concrete Card objects, separating object creation from object usage (high cohesion). This was useful because Card objects had to be created in several scenarios (for example, creating a player's Deck, or using the Master Summoner to create Air Elementals). The Factory Method makes Card construction resilient to project specification change. For example, if new cards are introduced, we can use our `CardFactory` to create that Card and provide it to the higher level classes. Additionally, let's say in the future we require parameters for Cards. In terms of actual card creation, the `CardFactory` can be modified to verify that the parameters are valid and return the Card. Without the Factory Method, we would

have to couple this logic into classes like Deck and Commands, which is inconvenient. So, our CardFactory makes our project more resilient to change.

Answers to Project Questions

How could you design activated abilities in your code to maximize code reuse?

We designed activated ability as an abstract class, with each unique activated ability having its own concrete class with its unique description/functionality. This ensures that the code which is common to any activated ability (ex. `getCost()`) is reused for each one and does not need to be repeated. For instance, "Deal 1 damage to target minion" will be implemented as a concrete subclass of activated ability. And each ability returns the command which executes the abilities actions. This allows us to easily extend to new abilities and reuse Commands such as `DamageMinionCommand` or `GainStatsCommand` which multiple abilities (triggered and activated) use.

What design pattern would be ideal for implementing enchantments? Why?

We implemented enchantments with decorators. The reasoning is that it makes stacking enchantments (modifications) on minions simple and handles any additional features to minions that need to be added for code flexibility. In our case, we implemented the enchantment functionality as minion decorators, so each enchantment will represent a unique decorator that can be applied to a base minion object.

Suppose we found a solution to the space limitations of the current user interface and wanted to allow minions to have any number and combination of activated and triggered abilities. What design patterns might help us achieve this while maximizing code reuse?

We currently have a triggered ability abstract class and an activated ability abstract class which inherit from an ability abstract class, with each unique ability being a concrete subclass. This ensures that the common functionalities between any two abilities will be shared. Additionally, the way abilities are called within the Minion class will be written in a way so that any ability can be called with the same method without affecting correctness. This ensures that code will not be duplicated. And we allow the minion to have a vector of unique pointers to Abilities and add a boolean of `is_triggered()` to differentiate between triggered abilities and activated abilities. The command pattern would also be able to handle ability actions.

All 3 of these answers were the same as to what we answered previously. We simply added on to answer 1.

Extra Credit Features

The extra credit feature we completed involved completing the entire project without memory leaks and without explicitly managing our own memory. This involved handling all memory with vectors and smart pointers, which was the requirement given in the guidelines. We were pretty comfortable working on this feature, given that this form of memory management was intuitive and easier to keep track of compared to explicit management. Any place where we would be allocating ownership/space for an object that would need to be deleted, we replaced with a `unique_ptr`, while returning the raw pointer for any getter methods.

Final Questions

What lessons did this project teach you about developing software in teams?

This project taught us that communication is the most important aspect to developing software in teams. All 3 of us had different coding practice styles and different setups that worked for us, so it was important that we would all be on the same page. Anytime one of us picked up on a task, we made it super clear what we were working on so nobody else would work on the same thing. Any important decisions related to the design would be communicated in our group chat, ensuring all of us were aware of any decisions being made. We set up a private github repository for the project, with each member working on a branch and opening a pull request for their changes, which the other 2 members would be able to open on their own and test the changes on before merging it with their own code. Through these standards that we had in place, we were able to work on the project asynchronously without blockers, which was very productive for our team.

What would you have done differently if you had the chance to start over?

In terms of workflow, there isn't much that we would have done differently for this project. Our team committed a lot of time to making design decisions and planning before DD2, which made the process of coding quite smooth. We also had a clear outline of what we should work on feature wise from the advice section of the game specification, which gave us a clear roadmap and a lot of confidence in what we were

building. From there, it involved working on tasks one by one and finishing the project ahead of time so we could test which we accomplished.

In terms of development, if we had the chance to start over and were given more time, one thing we would pay more attention to is the Interface Segregation Principle. The lack of time made us favour completion over ideal design in some cases, and interface segregation was one of them. This was evident in the Command class, which had full access to the Game object and could possibly manipulate more things about the game state than it needed to. If we were to attempt this project again, we would try to find a way to give Commands that modify a Minion only access to that Minion, and give Commands that modify a Player's stats only access to that Player, etc. In this way, we would preserve interface segregation.